
SettleIQ — AGENTS.md

Purpose

This file defines the rules and development principles that all AI coding agents must follow while working on SettleIQ.

The agent must treat the following documents as the authoritative project context:

docs/

├── 01_PRODUCT_SPEC.md

├── 02_SYSTEM_ARCHITECTURE.md

├── 03_DATA_SPEC.md

└── 04_IMPLEMENTATION_ROADMAP.md

Before making significant changes, the agent should understand these documents and follow them consistently.

---

1. Core Development Rule

Build only the current phase.

Do not attempt to implement the entire application at once.

The current development phase will be explicitly provided by the user.

If the current phase is unclear, ask before making large architectural changes.

---

2. Preserve the Existing Architecture

Follow the architecture defined in:

docs/02_SYSTEM_ARCHITECTURE.md

The project uses a:

Modular Monolithic Architecture

Do not introduce:

- **Microservices**
- **Kubernetes**
- **Message queues**
- **Distributed systems**
- **Multi-agent architectures**
- **Complex infrastructure**

unless explicitly requested.

---

3. Follow the Implementation Roadmap

Use:

docs/04_IMPLEMENTATION_ROADMAP.md

as the development sequence.

Do not skip foundational phases simply because a later feature appears easier or more interesting.

The intended sequence is:

Foundation

↓

Database

↓

Synthetic Data

↓

Data Validation

↓

Reconciliation

↓

Exceptions

↓

Evidence

↓

AI Investigation

↓

Guardrails

↓

Human Review

↓

Audit

↓

Frontend

↓

Evaluation

↓

Advanced Features

↓

Deployment

---

4. Do Not Overengineer

Prefer the simplest implementation that correctly solves the current requirement.

Before adding a dependency, framework, abstraction, or service, ask:

> **"Is this actually necessary for the current MVP?"**

Avoid unnecessary:

- **Dependencies**
- **Abstractions**
- **Design patterns**
- **Configuration layers**
- **Infrastructure**
- **APIs**
- **Database tables**
- **Components**

Simple and maintainable code is preferred over impressive-looking complexity.

---

5. Financial Logic Must Be Deterministic

Critical financial calculations must never depend on an LLM.

The following must be implemented using normal program logic:

- **Amount calculations**
- **Difference calculations**
- **Payment/settlement matching**
- **Fee calculations**
- **Duplicate detection**
- **Missing record detection**
- **Exception classification**
- **Confidence thresholds**
- **Guardrail conditions**
- **Resolution rules**

Core principle

> **Code calculates. AI investigates, reasons, and explains.**

---

6. AI Must Not Replace Deterministic Reconciliation

The LLM should not be asked to scan the entire financial dataset and determine what matches.

The correct flow is:

Financial Data

↓

Deterministic Reconciliation

↓

Exception Detected

↓

Exception Classification

↓

Evidence Builder

↓

AI Investigation

The AI receives a focused evidence package, not unrestricted database access.

---

7. AI Responsibilities

The AI may:

- **Investigate an identified exception**
- **Interpret related records**
- **Identify probable root cause**
- **Explain the discrepancy**
- **Assess confidence**
- **Recommend an action**
- **Reference supporting evidence**

The AI must not:

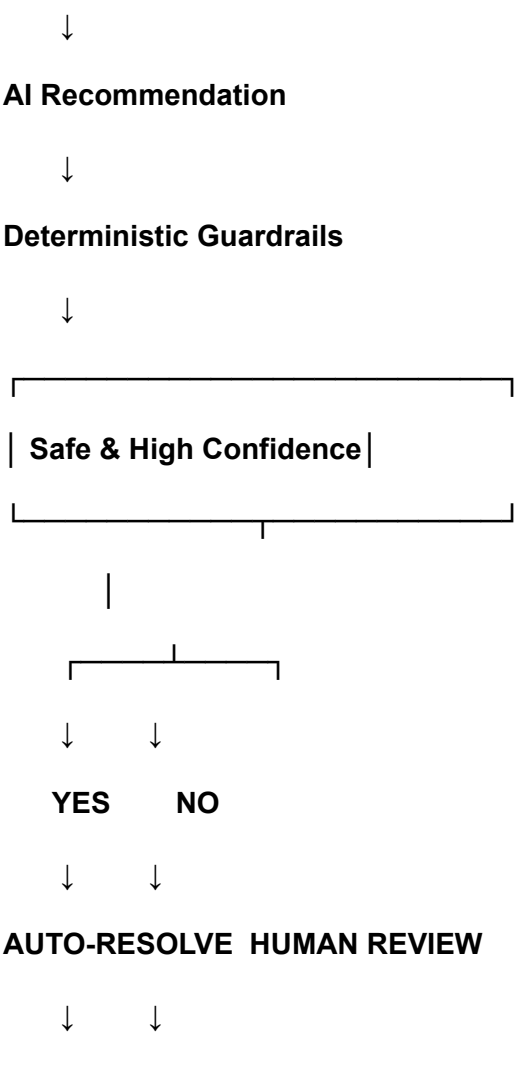
- **Directly modify financial records**
- **Directly mark transactions as resolved**
- **Perform critical financial calculations when deterministic code can do them**
- **Invent evidence**
- **Invent transaction records**
- **Access unrestricted database contents**
- **Make decisions outside its defined scope**

---

8. Guardrails Are Mandatory

AI recommendations must pass through deterministic guardrails before any automatic resolution occurs.

The intended flow is:



The human should not manually choose between auto-resolution and human review for every case.

The system should make that routing decision automatically based on predefined guardrails.

---

9. Auto-Resolution Rules

A case may be automatically resolved only when the predefined guardrails are satisfied.

Examples:

Confidence threshold passed

AND

Required evidence exists

AND

Known resolution rule satisfied

AND

No conflicting evidence exists

If these conditions are not satisfied:

→ HUMAN REVIEW

The AI itself must never directly perform the financial state change.

The actual state change should be performed by a controlled backend Resolution Service after guardrails approve it.

---

10. Human Review Is the Safety Layer

Cases should be sent to human review when:

- **Confidence is below the required threshold**
- **Evidence is incomplete**
- **The root cause is ambiguous**
- **No known resolution rule applies**
- **Conflicting records exist**
- **The AI response is invalid**
- **The AI service fails**
- **The system encounters an unexpected condition**

The goal is:

Automate safe cases and escalate uncertain cases.

---

11. AI Output Must Be Structured

Do not rely on free-form AI responses.

Prefer structured JSON.

Example:

```
{
  "exception_type": "AMOUNT_MISMATCH",
  "root_cause": "PROCESSING_FEE",
  "confidence": 0.96,
  "recommended_action": "AUTO_RESOLVE",
  "explanation": "The settlement is lower than the payment by exactly the recorded processing fee.",
  "evidence_ids": [
    "PAY_10001",
    "SET_70001",
    "FEE_90001"
  ]
}
```

Validate the AI response against a defined schema before using it.

---

12. Evidence-Based AI

Every AI conclusion must be based on the evidence provided to it.

The AI must not invent:

- **Amounts**
- **IDs**
- **Fees**

- **Settlement records**
- **Dates**
- **Transaction relationships**
- **Root causes**

If sufficient evidence is unavailable, the system should prefer:

HUMAN_REVIEW

rather than guessing.

---

13. Exception Types

The current supported exception categories are:

AMOUNT_MISMATCH

MISSING_SETTLEMENT

DUPLICATE

REFERENCE_MISMATCH

UNKNOWN / AMBIGUOUS

The first four are known exception categories.

`UNKNOWN / AMBIGUOUS` **is the fallback category for cases that cannot be confidently classified using known rules.**

Do not introduce additional exception categories without first discussing the requirement.

---

14. Database Rules

The database is the source of truth for application state.

Do not store important application state only in:

- **Frontend state**
- **Browser storage**
- **LLM conversation history**
- **Temporary variables**

Financial records and important decisions must be persisted appropriately.

---

15. API Rules

Keep API endpoints focused and predictable.

Follow REST conventions where practical.

Do not create duplicate endpoints that perform the same operation.

Validate incoming API data using appropriate schemas.

Return meaningful errors rather than exposing raw stack traces to users.

---

16. Error Handling

The application must fail safely.

Never allow an error to result in an incorrect automatic financial resolution.

Examples:

LLM unavailable

↓

Human Review

Invalid AI response

↓

Human Review

Missing evidence

↓

Human Review

Unexpected financial state

↓

Stop / Investigate

Errors should be logged appropriately.

---

17. Testing Requirements

Every meaningful piece of financial logic should have tests.

Prioritize tests for:

- **Payment/settlement matching**
- **Amount calculations**
- **Fee calculations**
- **Duplicate detection**
- **Missing settlement detection**
- **Exception classification**
- **Guardrail logic**
- **Auto-resolution conditions**
- **Human-review routing**
- **AI response validation**

Do not rely only on manual testing for critical financial logic.

---

18. Synthetic Data Must Have Ground Truth

All synthetic anomalies should have known expected outcomes.

The data generator must maintain ground truth so that system performance can be measured.

Example:

Transaction Expected Result

PAY_10001 MATCHED

PAY_10002 AMOUNT_MISMATCH

PAY_10003 MISSING_SETTLEMENT

PAY_10004 DUPLICATE

PAY_10005 REFERENCE_MISMATCH

Never modify ground truth simply to make the system's results look better.

---

19. Do Not Fake Metrics

Never fabricate:

- **Accuracy**
- **Precision**
- **Recall**
- **Auto-resolution rate**
- **Confidence**
- **Processing time**
- **Dataset size**
- **Evaluation results**

All metrics shown in the application, README, pitch, or submission must come from actual execution.

---

20. Do Not Fake Functionality

Do not create:

- **Fake AI responses**
- **Hardcoded dashboard numbers**
- **Fake database records presented as live data**
- **Mock success messages presented as real functionality**
- **Placeholder buttons that appear functional**

During early development, mocks may be used temporarily when explicitly required, but they must be clearly separated from production functionality and replaced before final submission.

---

21. Preserve Working Code

Before modifying an existing component:

1. **Understand what it currently does.**
2. **Check whether it is already working.**
3. **Make the smallest change necessary.**
4. **Run relevant tests.**
5. **Verify that existing functionality still works.**

Do not rewrite working modules simply to use a preferred coding style.

---

22. Avoid Scope Creep

Do not introduce unrelated features.

The core project is:

> **Multi-source settlement reconciliation + AI-powered exception investigation + bounded resolution + human review.**

Do not start building unrelated systems such as:

- **Payment processing**
- **Full accounting software**
- **Banking infrastructure**
- **Fraud detection platform**
- **Tax platform**
- **Complete ERP**
- **General-purpose AI assistant**

unless explicitly added to the project scope.

---

23. Advanced Features Come Later

Features such as:

- **Settlement Q&A**
- **Advanced analytics**
- **Exception trends**
- **What-if analysis**

should only be implemented after the core MVP is stable.

If time becomes limited:

Advanced Features

↓

Sacrifice First

Do not sacrifice the core reconciliation system for a flashy feature.

---

24. Code Quality

Code should be:

- **Readable**
- **Modular**
- **Consistent**
- **Well named**
- **Reasonably documented**
- **Easy for another developer to understand**

Avoid unnecessary comments that merely describe obvious code.

Comments should explain:

- **Why something is done**
- **Important business rules**
- **Non-obvious decisions**
- **Safety constraints**

---

25. Environment & Secrets

Never hardcode:

- **API keys**
- **Passwords**
- **Tokens**
- **Secrets**
- **Production credentials**

Use environment variables.

Example:

.env

must not be committed to Git.

Provide:

.env.example

containing only placeholder values.

---

26. Git Rules

Use Git regularly.

After completing a meaningful piece of work:

Implement

↓

Test

↓

Verify

↓

Commit

Use meaningful commit messages.

Examples:

feat: add payment data models

feat: implement reconciliation engine

feat: add exception classification

feat: add AI investigation service

fix: handle missing settlement records

test: add reconciliation rule tests

Do not make one enormous commit containing the entire project.

---

27. Documentation Rules

Update documentation when an important architectural or functional decision changes.

Do not create excessive documentation for small implementation details.

The main documentation remains:

01_PRODUCT_SPEC.md

02_SYSTEM_ARCHITECTURE.md

03_DATA_SPEC.md

04_IMPLEMENTATION_ROADMAP.md

AGENTS.md

---

28. When Requirements Are Ambiguous

Do not silently invent major requirements.

If a decision could materially affect:

- **Architecture**
- **Database design**
- **Financial logic**
- **AI behavior**
- **Security**
- **Product scope**

ask for clarification before implementing it.

For small implementation details, choose the simplest reasonable approach and document the decision.

---

29. Before Adding a New Dependency

Before installing a new package, consider:

1. **Is it actually necessary?**
2. **Can the existing stack solve the problem?**
3. **Does it increase deployment complexity?**
4. **Is it reliable and maintained?**
5. **Will it make the project harder to explain?**

Prefer the existing stack whenever possible.

---

30. Current Project Priorities

When there is a conflict between features, prioritize in this order:

1. Correctness

↓

2. Reliability

↓

3. Safety

↓

4. Testability

↓

5. Usability

↓

6. Performance

↓

7. Visual polish

↓

8. Advanced features

A visually impressive system that produces incorrect financial results is unacceptable.

---

31. Agent Reporting Format

After completing a development task, the agent should report:

Changes Made

What was implemented.

Files Changed

List the files created or modified.

Tests Run

List tests or verification performed.

Result

Whether the implementation works.

Issues

Any known limitations, errors, or unresolved problems.

Next Step

State what the next roadmap phase/task would be — but do not implement it unless instructed.

---

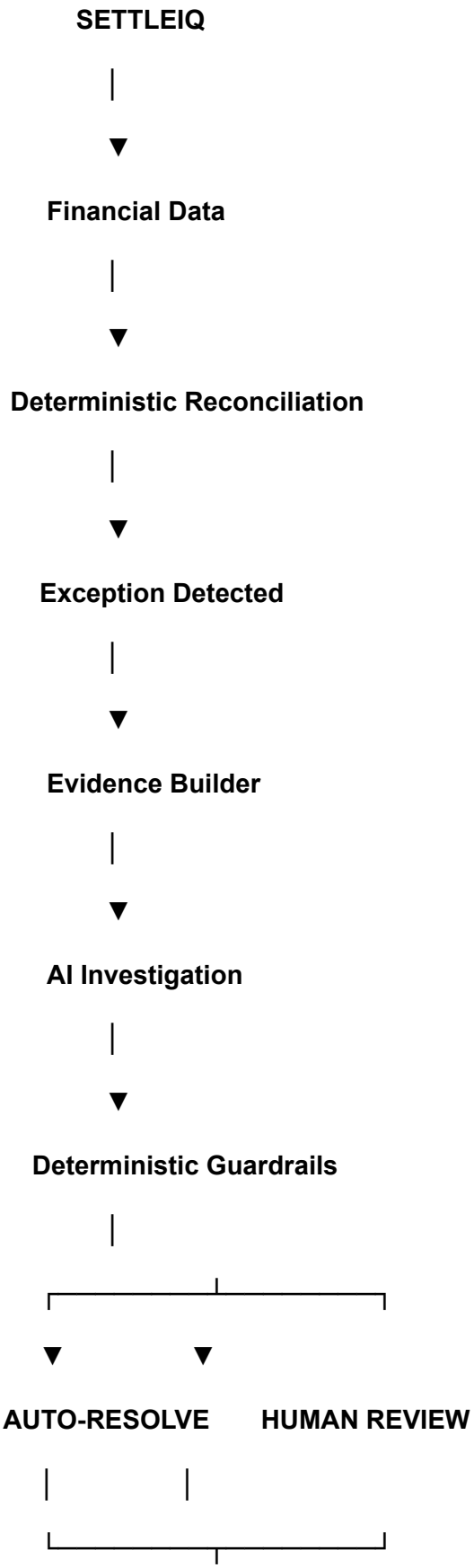
32. Final Rule

The most important rule:

> **Do not optimize for writing the most code. Optimize for building the most reliable working product.**

SettleIQ should demonstrate that AI is being used where it provides genuine value while deterministic software remains responsible for financial correctness and safety.

The target architecture is:



▼

Audit Trail

|

▼

Dashboard

Core philosophy:

> **Deterministic systems establish what happened. AI investigates why. Guardrails determine whether automation is safe. Humans handle uncertainty.**